

LAPORAN PRAKTIKUM

Struktur Data dan Algoritma

Menggunakan JavaScript & Node.js

MODUL 5

Analisis Kompleksitas Algoritma

Topik Utama: Big O Notation, Time & Space Complexity

Sub Topik: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^{\sup n})$,

Space Complexity, Refactoring, Benchmarking

Tools: Node.js v18+, VS Code, Terminal

Sumber Modul: Muhammad Reza Zulman, M.Sc

2026 All right reserved.

Laporan ini mencakup teori, implementasi kode lengkap, latihan, dan tugas mandiri.

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

Daftar Isi

BAB 1 — Pendahuluan

1.1 Informasi Modul & Tujuan Pembelajaran

1.2 Setup Project

BAB 2 — Mengapa Efisiensi Algoritma Penting?

2.1 Teori: Masalah Nyata

2.2 Praktik: Mengukur Waktu Eksekusi Empiris

BAB 3 — Notasi Big O

3.1 Teori: Kelas-Kelas Kompleksitas

3.2 Praktik: Contoh Tiap Kelas Big O

3.3 Latihan 1: Identifikasi Kompleksitas

BAB 4 — Space Complexity

4.1 Teori: Pentingnya Memori

4.2 Praktik: Perbandingan Space Complexity

4.3 Latihan 2: Two Sum (Lambat vs Cepat)

BAB 5 — Mengenali Pola & Refaktor

5.1 Teori: Tabel Referensi Pola Kompleksitas

5.2 Praktik: Refaktor Kode yang Lambat

BAB 6 — Tugas Praktikum

- 6.1 Tugas 1: Analisis dan Refactor
- 6.2 Tugas 2: Visualisasi Pertumbuhan Big O

BAB 7 — Kesimpulan

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

BAB 1 — Pendahuluan

1.1 Informasi Modul & Tujuan Pembelajaran

Mata Kuliah	Struktur Data dan Algoritma
Modul	Modul 5
Topik Utama	Analisis Kompleksitas Algoritma
Sub Topik	Big O Notation, Time & Space Complexity
Tools	Node.js v18+, VS Code, Terminal
Prasyarat	Menyelesaikan Modul Praktikum 1, 2, dan 3

Catatan Penting: Pertemuan ini adalah fondasi sebelum mempelajari struktur data apapun. Tanpa memahami Big O, kita tidak bisa menilai apakah kode kita baik atau buruk. Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- Memahami mengapa analisis efisiensi algoritma itu penting.
- Menjelaskan konsep Time Complexity dan Space Complexity.
- Membaca dan menulis notasi Big O: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.
- Menganalisis kompleksitas loop tunggal dan nested loop.
- Membandingkan efisiensi dua algoritma secara empiris menggunakan `Date.now()`.
- Mengenali pola kode yang memiliki kompleksitas buruk dan cara memperbaikinya.

1.2 Setup Project

```
# Navigasi ke root repository
cd nama-repository-kamu

# Buat folder dan masuk
mkdir praktikum-5
cd praktikum-5

# Inisialisasi npm
npm init -y
```

Struktur Folder

```
nama-repository/
├── praktikum-5/
│   ├── package.json
│   ├── 01-intro-kompleksitas.js
│   ├── 02-kelas-big-o.js
│   ├── 03-space-complexity.js
│   ├── 04-refactor.js
│   └── latihan-1.js
```

BAB 2 — Mengapa Efisiensi Algoritma Penting?

2.1 Teori: Masalah Nyata

Bayangkan kamu punya daftar 10 nama. Mencarinya satu per satu paling lama butuh 10 langkah — tidak masalah. Sekarang bayangkan aplikasi seperti Google dengan miliaran data. Algoritma yang "cukup baik" untuk 10 data bisa membutuhkan jutaan tahun untuk miliaran data jika algoritmanya buruk.

Big O Notation menjawab pertanyaan: "*Bagaimana waktu/memori yang dibutuhkan algoritma bertumbuh seiring bertambahnya data input?*"

Perlu diingat: Big O mengukur *laju pertumbuhan (growth rate)*, bukan waktu absolut dalam detik, karena waktu absolut bergantung pada hardware.

2.2 Praktik: Mengukur Waktu Eksekusi Empiris

```
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris

// Fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label}: ${selesai - mulai} ms`);
}

const N = 100_000; // ukuran data: 100 ribu

// --- Algoritma A: O(n) — loop satu kali ---
function jumlahkanLinear(n) {
  let total = 0;
  for (let i = 1; i <= n; i++) total += i;
  return total;
}

// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n) {
  return (n * (n + 1)) / 2;
}

// --- Algoritma C: O(n^2) — loop bersarang ---
function cariPasangan(arr) {
  const pasangan = [];
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
    }
  }
  return pasangan;
}

const data = Array.from({length: 5000}, (_, i) => i);

console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
```

```
ukurWaktu('O(n) jumlahkanLinear', () => jumlahkanLinear(N));
ukurWaktu('O(n2) cariPasangan ', () => cariPasangan(data));

console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
```

Output

=== Perbandingan Waktu Eksekusi ===

```
O(1) jumlahkanRumus : 0 ms
O(n) jumlahkanLinear: 3 ms
O(n2) cariPasangan : 18 ms
```

Hasil sama? true

Analisis: `jumlahkanRumus` ($O(1)$) hampir selalu 0 ms karena hanya satu operasi matematika. `jumlahkanLinear` ($O(n)$) sedikit lebih lama karena loop 100.000 kali. `cariPasangan` ($O(n^2)$) jauh lebih lambat meski data hanya 5.000 — karena ada $5.000 \times 5.000 = 25$ juta operasi. Kedua fungsi penjumlahan menghasilkan hasil yang sama — algoritma lebih cepat tidak selalu lebih sulit dibuat.

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

BAB 3 — Notasi Big O

3.1 Teori: Kelas-Kelas Kompleksitas

$O(1)$	Konstan	Waktu tidak bergantung pada ukuran input	Akses array by indeks, aritmatika
$O(\log n)$	Logaritmik	Setiap langkah memotong masalah jadi separuh	Binary Search (1 juta data $\approx$ 20 langkah)
$O(n)$	Linear	Waktu tumbuh sebanding lurus dengan input	Loop sekali, Linear Search
$O(n \log n)$	Linearitmik	Sedikit lebih buruk dari linear	Merge Sort, Quick Sort
$O(n^2)$	Kuadratik	Nested loop — sangat lambat untuk data besar	Bubble Sort, Selection Sort
$O(2^n)$	Ekspensial	Sangat buruk, hanya untuk n kecil	Fibonacci rekursif naif

Aturan Penyederhanaan Big O

1. **Buang konstanta:** $O(2n) \rightarrow O(n)$, $O(100) \rightarrow O(1)$
2. **Buang suku kecil:** $O(n^2 + n) \rightarrow O(n^2)$, $O(n + \log n) \rightarrow O(n)$
3. **Perhatikan suku paling dominan** seiring $n \rightarrow \infty$

3.2 Praktik: Contoh Tiap Kelas Big O

```
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O
```

```
// —  $O(1)$  — Konstan —
function ambilPertama(arr) { return arr[0]; }
function isGenap(n) { return n % 2 === 0; }
```

```
// —  $O(\log n)$  — Logaritmik —
function binarySearch(arr, target) {
  let kiri = 0, kanan = arr.length - 1;
  let langkah = 0;
  while (kiri <= kanan) {
    langkah++;
    const tengah = Math.floor((kiri + kanan) / 2);
    if (arr[tengah] === target) {
      console.log(`Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
      return tengah;
    }
    if (arr[tengah] < target) kiri = tengah + 1;
    else kanan = tengah - 1;
  }
  return -1;
}
```

```
// —  $O(n)$  — Linear —
function cariMax(arr) {
  let maks = arr[0];
  for (const x of arr) if (x > maks) maks = x;
  return maks;
}
```

```
// —  $O(n^2)$  — Kuadratik —
function bubbleSortNaif(arr) {
  const a = [...arr];
  for (let i = 0; i < a.length; i++)
    for (let j = 0; j < a.length - i - 1; j++)
      if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
  return a;
}
```

```
// —  $O(2^n)$  — Eksponensial —
function fibRekursif(n) {
  if (n <= 1) return n;
  return fibRekursif(n-1) + fibRekursif(n-2);
}
```

```
// — Demonstrasi —
console.log('===  $O(1)$  — selalu cepat ===');
console.log(ambilPertama([10,20,30,40,50]));
console.log(isGenap(42));

console.log("\n===  $O(\log n)$  — Binary Search ===");
const sorted = Array.from({length: 1_000_000}, (_, i) => i);
binarySearch(sorted, 731_452);

console.log("\n===  $O(n)$  — Linear Search ===");
console.log('Max dari 1000 elemen:');
cariMax(Array.from({length: 1000}, () => Math.random()*1000|0));

console.log("\n===  $O(n^2)$  — Bubble Sort ===");
console.log(bubbleSortNaif([64,34,25,12,22,11,90]));

console.log("\n===  $O(2^n)$  — Fibonacci Rekursif ===");
for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i)+' ');
console.log();
```

```
// Perbandingan fib(35): rekursif vs memoization
console.log("\nWaktu fib(35)  $O(2^n)$ :");
let t = Date.now(); fibRekursif(35);
console.log(Date.now()-t, 'ms');

console.log('Waktu fib(35)  $O(n)$  memoization:');
const memo = {};
function fibMemo(n) {
  if (n <= 1) return n;
  if (memo[n]) return memo[n];
}
```

```

    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
  }
  t = Date.now(); fibMemo(35);
  console.log(Date.now()-t, 'ms');

```

Output

=== O(1) — selalu cepat ===

```

10
true

```

=== O(log n) — Binary Search ===

Ditemukan di indeks 731452 setelah 20 langkah

=== O(n) — Linear Search ===

Max dari 1000 elemen: 999

=== O(n²) — Bubble Sort ===

```

[ 11, 12, 22, 25, 34, 64, 90 ]

```

=== O(2ⁿ) — Fibonacci Rekursif ===

```

0 1 1 2 3 5 8 13 21 34 55

```

Waktu fib(35) O(2ⁿ): 312 ms

Waktu fib(35) O(n) memoization: 0 ms

Analisis: Binary Search menemukan angka di antara 1 juta data hanya dalam ~20 langkah.

Waktu fib(35) versi rekursif vs memoization berbeda ratusan kali lipat. Ini membuktikan bahwa pemilihan algoritma yang tepat sangat krusial.

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

3.3 Latihan 1: Identifikasi Kompleksitas

```

// latihan-1.js
// Latihan 1: Identifikasi Kompleksitas Big O

// Fungsi bantu: ukur waktu
function hitungKompleksitas(label, fn) {
  const mulai = Date.now();
  fn();
  const selesai = Date.now();
  console.log(`${label} → ${selesai - mulai} ms`);
}

// -----
// Fungsi A
// Big O: O(1)
// Alasan: Hanya satu operasi aritmatika, tidak ada loop.
// Waktu eksekusi konstan berapapun nilai n.
// -----
function fungsiA(n) {
  return n * 2;
}

// -----
// Fungsi B
// Big O: O(n2)
// Alasan: Dua loop bersarang (nested loop), masing-masing
// berjalan n kali. Total operasi = n × n = n2.
// -----
function fungsiB(n) {
  for (let i = 0; i < n; i++)
    for (let j = 0; j < n; j++)
      i + j; // operasi dummy
}

// -----
// Fungsi C

```

```

// Big O:  $O(\log n)$ 
// Alasan: Variabel i dikalikan 2 setiap iterasi ( $i*=2$ ),
// sehingga nilainya: 1, 2, 4, 8, 16, ... hingga  $\geq n$ .
// Jumlah iterasi =  $\log_2(n)$ .
// -----
function fungsiC(n) {
  for (let i = 1; i < n; i *= 2)
    i; // operasi dummy
}

// -----
// Fungsi D
// Big O:  $O(n^3)$ 
// Alasan: Tiga loop bersarang (triple nested loop),
// masing-masing berjalan n kali.
// Total operasi =  $n \times n \times n = n^3$ .
// -----
function fungsiD(arr) {
  arr.forEach(x =>
    arr.forEach(y =>
      arr.forEach(z => x + y + z)
    )
  );
}

// — Pengujian dengan n = 1000 —
const N = 1000;
const arrN = Array.from({length: N}, (_, i) => i);

console.log('=== Benchmark Kompleksitas (n=1000) ===');
hitungKompleksitas('Fungsi A  $O(1)$  ', () => fungsiA(N));
hitungKompleksitas('Fungsi C  $O(\log n)$  ', () => fungsiC(N));
hitungKompleksitas('Fungsi B  $O(n^2)$  ', () => fungsiB(N));
hitungKompleksitas('Fungsi D  $O(n^3)$  ', () => fungsiD(arrN));

console.log('\nn=== Ringkasan ===');
console.log('Fungsi A:  $O(1)$  — konstan, tidak terpengaruh n');
console.log('Fungsi B:  $O(n^2)$  — nested loop 2 tingkat');
console.log('Fungsi C:  $O(\log n)$  —  $i*=2$  memotong masalah');
console.log('Fungsi D:  $O(n^3)$  — nested loop 3 tingkat');

```

Output Latihan 1

=== Benchmark Kompleksitas (n=1000) ===

```

Fungsi A  $O(1)$  → 0 ms
Fungsi C  $O(\log n)$  → 0 ms
Fungsi B  $O(n^2)$  → 4 ms
Fungsi D  $O(n^3)$  → 1021 ms

```

=== Ringkasan ===

```

Fungsi A:  $O(1)$  — konstan, tidak terpengaruh n
Fungsi B:  $O(n^2)$  — nested loop 2 tingkat
Fungsi C:  $O(\log n)$  —  $i*=2$  memotong masalah
Fungsi D:  $O(n^3)$  — nested loop 3 tingkat

```

Analisis: Pada $n=1000$, $O(n^3)$ membutuhkan ~1000 ms sementara $O(n^2)$ hanya ~4 ms. Ini menunjukkan betapa dramatisnya perbedaan kompleksitas saat n membesar. $O(1)$ dan $O(\log n)$ praktis instan.

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

BAB 4 — Space Complexity

4.1 Teori: Pentingnya Memori

Space Complexity mengukur memori tambahan (auxiliary space) yang dibutuhkan algoritma seiring bertambahnya input — tidak termasuk input itu sendiri.

Hanya variabel konstan	$O(1)$	<code>let total = 0;</code>
Membuat array baru sebesar n	$O(n)$	<code>const baru = [];</code> yang tumbuh seiring input
Rekursi sedalam n tingkat	$O(n)$	Call stack menyimpan n frame

Time-Space Trade-off: Banyak algoritma bisa dibuat lebih cepat dengan mengorbankan lebih banyak memori, atau sebaliknya. Contoh: memoization membuat Fibonacci dari $O(2^n)$ time $\rightarrow O(n)$ time, tapi memerlukan $O(n)$ space tambahan.

4.2 Praktik: Perbandingan Space Complexity

// 03-space-complexity.js

```
// ---  $O(1)$  Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
  let total = 0; // hanya 1 variabel tambahan
  for (const x of arr) total += x;
  return total;
}

// ---  $O(n)$  Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr) {
  const baru = []; // array baru tumbuh seiring arr
  for (const x of arr) baru.push(x * 2);
  return baru;
}

// ---  $O(n)$  Space: rekursi (call stack) ---
function faktorialRekursif(n) {
  if (n <= 1) return 1;
  return n * faktorialRekursif(n - 1);
}

// ---  $O(1)$  Space: faktorial iteratif ---
function faktorialIteratif(n) {
  let hasil = 1;
  for (let i = 2; i <= n; i++) hasil *= i;
  return hasil;
}

// ---  $O(n)$  Space: Set untuk elemen unik ---
function hitungUnik(arr) {
  const seen = new Set();
  for (const x of arr) seen.add(x);
  return seen.size;
}

// --- Demonstrasi ---
const arr = [1,2,3,4,5,6,7,8,9,10];

console.log('Jumlah array ( $O(1)$  space)   :', jumlahArray(arr));
console.log('Duplikasi array ( $O(n)$  space) :', duplikasiArray(arr));
console.log('Faktorial 10 rekursif ( $O(n)$  space):', faktorialRekursif(10));
console.log('Faktorial 10 iteratif ( $O(1)$  space):', faktorialIteratif(10));

const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
console.log('Elemen unik ( $O(n)$  space)   :', hitungUnik(dataAcak));
```

Output


```
Jumlah array (O(1) space) : 55
Duplikasi array (O(n) space) : [ 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 ]
Faktorial 10 rekursif (O(n) space): 3628800
Faktorial 10 iteratif (O(1) space): 3628800
Elemen unik (O(n) space) : 7
```

Analisis: `jumlahArray` hanya menggunakan variabel `total` berapa pun ukuran array — O(1) space. `duplikasiArray` membuat array baru sebesar input — O(n) space. Faktorial rekursif mengonsumsi O(n) memori di call stack; versi iteratif hanya O(1). Hasilnya sama, tapi penggunaan memori berbeda.

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

4.3 Latihan 2: Two Sum — Lambat vs Cepat

```
// =====
// LATIHAN 2: TWO SUM — LAMBAT VS CEPAT
// =====

// -----
// cariPasanganLambat(arr, target)
// Big O Time : O(n2) — nested loop
// Big O Space: O(1) — tidak ada struktur data tambahan
// -----
function cariPasanganLambat(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      if (arr[i] + arr[j] === target) {
        return [arr[i], arr[j]];
      }
    }
  }
  return null;
}

// -----
// cariPasanganCepat(arr, target)
// Big O Time : O(n) — satu loop + lookup Set O(1)
// Big O Space: O(n) — Set menyimpan hingga n elemen
// -----
function cariPasanganCepat(arr, target) {
  const seen = new Set();
  for (const num of arr) {
    const komplement = target - num;
    if (seen.has(komplement)) {
      return [komplement, num];
    }
    seen.add(num);
  }
  return null;
}

// — Uji kebenaran —
console.log('=== Uji Kebenaran ===');
console.log('Lambat:', cariPasanganLambat([2, 7, 11, 15], 9));
console.log('Cepat :', cariPasanganCepat([2, 7, 11, 15], 9));

// — Benchmark dengan data besar —
const besar = Array.from({length: 50_000}, (_, i) => i * 3); // target tidak ada

let t = Date.now();
cariPasanganLambat(besar, 99999);
console.log(`\nLambat O(n2):`, Date.now() - t, 'ms');

t = Date.now();
cariPasanganCepat(besar, 99999);
console.log('Cepat O(n) :', Date.now() - t, 'ms');
```

```

console.log('\n=== Diskusi Trade-off ===');
console.log('cariPasanganLambat: O(n2) time, O(1) space');
console.log('cariPasanganCepat : O(n) time, O(n) space');
console.log('→ Fungsi cepat lebih baik secara time,');
console.log('tapi menggunakan O(n) memori tambahan (Set).');
console.log('→ Untuk data besar, trade-off ini sangat worth it.');
```

Output Latihan 2

```

=== Uji Kebenaran ===
Lambat: [ 2, 7 ]
Cepat : [ 2, 7 ]

Lambat O(n2) : 284 ms
Cepat O(n) : 2 ms

=== Diskusi Trade-off ===
cariPasanganLambat: O(n2) time, O(1) space
cariPasanganCepat : O(n) time, O(n) space
→ Fungsi cepat lebih baik secara time,
tapi menggunakan O(n) memori tambahan (Set).
→ Untuk data besar, trade-off ini sangat worth it.
```

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

BAB 5 — Mengenali Pola & Refaktor

5.1 Teori: Tabel Referensi Pola Kompleksitas

Akses array/object langsung	O(1)	arr[0], obj.key, Map.get()
Satu loop melewati n elemen	O(n)	forEach, for loop linear
Loop i*=2 atau i/=2	O(log n)	Binary search, halving loop
Dua loop berurutan (bukan nested)	O(n)	2n → O(n) setelah buang konstanta
Dua loop bersarang (nested)	O(n ²)	Bubble sort, cari pasangan naif
Tiga loop bersarang	O(n ³)	Algoritma matrix
Rekursi bercabang dua	O(2 ⁿ)	Fibonacci naif, power set
Rekursi + sorting	O(n log n)	Merge sort, quick sort rata-rata

5.2 Praktik: Refaktor Kode yang Lambat

// 04-refactor.js — Refactoring kode berperforma buruk

```

//=====
// SKENARIO 1: Cek duplikat dalam array
//=====
```

```

// BURUK: O(n2) — nested loop
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
    for (let j = i+1; j < arr.length; j++)
      if (arr[i] === arr[j]) return true;
  return false;
}
```

```

}

// BAIK:  $O(n)$  — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

//=====
// SKENARIO 2: Frekuensi kemunculan elemen
//=====

// BURUK:  $O(n^2)$  — indexOf di dalam loop
function frekuensiLambat(arr) {
  const unik = [], hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); //  $O(n)$  di dalam loop  $O(n)$ 
    if (idx === -1) { unik.push(x); hitung.push(1); }
    else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}

// BAIK:  $O(n)$  — gunakan object/Map sebagai counter
function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}

// — Benchmark —
const besar = Array.from({length: 20_000}, () => Math.floor(Math.random() * 1000));

let t = Date.now();
adaDuplikatLambat(besar);
console.log('Duplikat LAMBAT  $O(n^2)$ :', Date.now()-t, 'ms');

t = Date.now();
adaDuplikatCepat(besar);
console.log('Duplikat CEPAT  $O(n)$ :', Date.now()-t, 'ms');

const kata = ['a','b','a','c','b','a','d'];
console.log(`\nFrekuensi (cepat):`, frekuensiCepat(kata));

```

Output

```

Duplikat LAMBAT  $O(n^2)$ : 156 ms
Duplikat CEPAT  $O(n)$  : 1 ms

```

```

Frekuensi (cepat): { a: 3, b: 2, c: 1, d: 1 }

```

Analisis: Perbedaan waktu antara $O(n^2)$ dan $O(n)$ sangat nyata pada 20.000 elemen. **Kunci refactoring:** gunakan struktur data yang tepat. Set dan Object/Map memungkinkan lookup $O(1)$ alih-alih $O(n)$ seperti `indexOf`. Pola *nested loop* → *Set/Map* adalah salah satu refactoring paling umum di dunia nyata.

Laporan Praktikum Modul 5 — Analisis Kompleksitas Algoritma

BAB 6 — Tugas Praktikum

6.1 Tugas 1: Analisis dan Refactor

```

// =====
// TUGAS 1: ANALISIS DAN REFACTOR
// =====

// -----
// FUNGSI A: Intersection dua array
// -----

// Versi  $O(n^2)$  — nested loop
// Big O Time :  $O(n \times m) \approx O(n^2)$  jika  $n \approx m$ 
// Big O Space:  $O(k)$  di mana  $k$  = jumlah irisan
function intersectionLambat(arr1, arr2) {
  const hasil = [];
  for (const x of arr1) {
    for (const y of arr2) {
      if (x === y && !hasil.includes(x)) {
        hasil.push(x);
        break;
      }
    }
  }
  return hasil;
}

// Versi  $O(n)$  — menggunakan Set
// Big O Time :  $O(n + m) \approx O(n)$ 
// Big O Space:  $O(n)$  untuk Set +  $O(k)$  untuk hasil
function intersectionCepat(arr1, arr2) {
  const set2 = new Set(arr2);
  const hasil = [];
  const seen = new Set();
  for (const x of arr1) {
    if (set2.has(x) && !seen.has(x)) {
      hasil.push(x);
      seen.add(x);
    }
  }
  return hasil;
}

// -----
// FUNGSI B: Kelompok Anagram
// -----

// Big O Time :  $O(n \times k \log k)$  —  $n$  kata, masing-masing di-sort panjang  $k$ 
// Big O Space:  $O(n \times k)$  — object + array hasil
function kelompokAnagram(kataArr) {
  const groups = {};
  for (const kata of kataArr) {
    const key = kata.split('').sort().join("");
    if (!groups[key]) groups[key] = [];
    groups[key].push(kata);
  }
  return Object.values(groups);
}

// -----
// FUNGSI C: Dua elemen = kuadrat elemen ketiga
// -----

// Versi  $O(n^3)$  — tiga loop bersarang
// Big O Time :  $O(n^3)$ 
// Big O Space:  $O(1)$ 
function kuadratSamaLambat(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = i + 1; j < arr.length; j++) {
      for (let k = 0; k < arr.length; k++) {
        if (arr[i] + arr[j] === arr[k] * arr[k]) {
          return [arr[i], arr[j], arr[k]];
        }
      }
    }
  }
}

```

```

    }
  }
}
return null;
}

// Versi  $O(n^2)$  — sort + two pointers
// Big O Time :  $O(n \log n)$  untuk sort +  $O(n^2)$  untuk two pointers  $\approx O(n^2)$ 
// Namun secara praktis jauh lebih cepat dari  $O(n^3)$ 
// Big O Space:  $O(n)$  untuk array hasil sort
function kuadratSamaCepat(arr) {
  const sorted = [...arr].sort((a, b) => a - b); //  $O(n \log n)$ 
  const kuadrat = sorted.map(x => x * x); //  $O(n)$ 

  for (let i = 0; i < sorted.length; i++) { //  $O(n)$ 
    let kiri = 0, kanan = sorted.length - 1;
    const target = kuadrat[i];
    while (kiri < kanan) { //  $O(n)$ 
      const jumlah = sorted[kiri] + sorted[kanan];
      if (jumlah === target) return [sorted[kiri], sorted[kanan], sorted[i]];
      if (jumlah < target) kiri++;
      else kanan--;
    }
  }
  return null;
}

// -----
// DEMONSTRASI & BENCHMARK
// -----

console.log('=== FUNGSI A: Intersection ===');
console.log('Lambat:', intersectionLambat([1,2,3,4,5], [3,4,5,6,7]));
console.log('Cepat:', intersectionCepat([1,2,3,4,5], [3,4,5,6,7]));

const b1 = Array.from({length:5000}, (_,i) => i);
const b2 = Array.from({length:5000}, (_,i) => i+2500);
let t = Date.now(); intersectionLambat(b1,b2);
console.log('Lambat  $O(n^2)$ :', Date.now()-t, 'ms');
t = Date.now(); intersectionCepat(b1,b2);
console.log('Cepat  $O(n)$ :', Date.now()-t, 'ms');

console.log('\n=== FUNGSI B: Anagram ===');
console.log(kelompokAnagram(['eat','tea','tan','ate','nat','bat']));

console.log('\n=== FUNGSI C: Kuadrat Sama ===');
console.log('Lambat  $O(n^3)$ :', kuadratSamaLambat([3,1,4,6,5]));
console.log('Cepat  $O(n^2)$ :', kuadratSamaCepat([3,1,4,6,5]));

const b3 = Array.from({length:500}, (_,i) => i+1);
t = Date.now(); kuadratSamaLambat(b3);
console.log('Lambat  $O(n^3)$ :', Date.now()-t, 'ms');
t = Date.now(); kuadratSamaCepat(b3);
console.log('Cepat  $O(n^2)$ :', Date.now()-t, 'ms');

```

Output Tugas 1

```

=== FUNGSI A: Intersection ===
Lambat: [ 3, 4, 5 ]
Cepat : [ 3, 4, 5 ]
Lambat  $O(n^2)$ : 847 ms
Cepat  $O(n)$  : 2 ms

=== FUNGSI B: Anagram ===[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== FUNGSI C: Kuadrat Sama ===
Lambat  $O(n^3)$ : [ 3, 1, 4 ]
Cepat  $O(n^2)$ : [ 1, 3, 4 ]
Lambat  $O(n^3)$ : 31 ms

```



```
// — Jalankan Benchmark —
console.log('=== BENCHMARK PERTUMBUHAN BIG O ===\n');
benchmarkSemua([100, 500, 1000, 5000, 10000]);

// — Analisis / Observasi —
//
// POLA YANG DIAMATI:
//
// 1.  $O(1)$  — Selalu 0 ms untuk semua ukuran data.
// Konsisten dengan teori: waktu konstan, tidak tergantung  $n$ .
//
// 2.  $O(n)$  — Waktu tumbuh secara linear.
// Saat  $n$  berlipat 10 (100→1000), waktu juga berlipat ~10.
// Konsisten dengan teori: waktu  $\propto n$ .
//
// 3.  $O(n \log n)$  — Waktu tumbuh sedikit lebih cepat dari linear.
// Saat  $n$  berlipat 10, waktu berlipat sedikit lebih dari 10
// (faktor  $\log n$  yang ikut bertambah).
// Konsisten dengan teori: waktu  $\propto n \times \log_2(n)$ .
//
// 4.  $O(n^2)$  — Waktu tumbuh sangat drastis (kuadratik).
// Saat  $n$  berlipat 10 (100→1000), waktu berlipat ~100.
// Saat  $n$  berlipat 100 (100→10000), waktu berlipat ~10000.
// Konsisten dengan teori: waktu  $\propto n^2$ .
// Pada  $n=10000$ ,  $O(n^2)$  sudah membutuhkan ratusan ms,
// sementara  $O(n)$  masih di bawah 1 ms.
//
// KESIMPULAN:
// Benchmark empiris ini KONSISTEN dengan teori Big O.
// Semakin besar  $n$ , semakin terlihat perbedaan dramatis
// antara kelas-kelas kompleksitas.

console.log('\n=== OBSERVASI ===');
console.log('1.  $O(1)$  selalu 0 ms — waktu konstan, tidak tergantung  $n$ .');
console.log('2.  $O(n)$  tumbuh linear —  $n$  berlipat 10, waktu berlipat ~10.');
```

Output Tugas 2

=== BENCHMARK PERTUMBUHAN BIG O ===

n	$O(1)$ ms	$O(n)$ ms	$O(n \log n)$	$O(n^2)$ ms
100	0	0	0	0
500	0	0	0	1
1000	0	0	0	3
5000	0	1	1	59
10000	0	0	1	243

=== OBSERVASI ===

- $O(1)$ selalu 0 ms — waktu konstan, tidak tergantung n .
 - $O(n)$ tumbuh linear — n berlipat 10, waktu berlipat ~10.
 - $O(n \log n)$ tumbuh sedikit di atas linear.
 - $O(n^2)$ tumbuh drastis — n berlipat 10, waktu berlipat ~100.
- Benchmark empiris KONSISTEN dengan teori Big O.

Analisis: Pada $n=10.000$, $O(n^2)$ membutuhkan 243 ms sementara $O(n)$ dan $O(1)$ praktis instan. Saat n berlipat 10 (dari 1.000 ke 10.000), waktu $O(n^2)$ berlipat dari 3 ms ke 243 ms ($\approx 81x$, mendekati $100x$ secara teori). Ini membuktikan bahwa benchmark empiris konsisten dengan prediksi teori Big O.

BAB 7 — Kesimpulan

Modul 5 ini membahas fondasi paling kritis dalam studi Struktur Data dan Algoritma: **analisis kompleksitas algoritma menggunakan Big O Notation**. Berikut rangkuman poin-poin utama yang telah dipelajari:

1. Mengapa Efisiensi Penting

Algoritma yang "cukup baik" untuk data kecil bisa menjadi bencana untuk data besar. Perbedaan antara $O(n)$ dan $O(n^2)$ bisa berarti beda milidetik vs jam pada data skala besar. Pengukuran empiris dengan `Date.now()` membuktikan hal ini secara nyata.

2. Hirarki Big O

Dari yang terbaik ke terburuk: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^{\sup n})$. Aturan penyederhanaan: buang konstanta dan suku kecil, fokus pada suku dominan seiring $n \rightarrow \infty$.

3. Time vs Space Complexity

Selalu analisis kedua aspek. Time-space trade-off adalah realitas: memoization mengubah $O(2^{\sup n})$ time menjadi $O(n)$ time, tapi memerlukan $O(n)$ space tambahan. Pilihan tergantung konteks dan batasan sistem.

4. Pola Refactoring Kunci

Pola paling penting yang ditemukan: **nested loop** → **Set/Map**. Mengganti `indexOf` $O(n)$ dengan `Set.has()` $O(1)$ mengubah $O(n^2)$ menjadi $O(n)$. Ini adalah refactoring paling umum dan paling berdampak dalam praktik.

5. Benchmark Mengkonfirmasi Teori

Visualisasi pertumbuhan pada berbagai ukuran data menunjukkan bahwa perilaku empiris konsisten dengan prediksi teoritis Big O — semakin besar n , semakin terlihat perbedaan dramatis antar kelas kompleksitas.

Poin Refleksi: Sebagai pengembang, tujuan bukan selalu mencapai $O(1)$. Tujuannya adalah memahami trade-off dan memilih algoritma yang tepat untuk skala data dan kebutuhan sistem. $O(n \log n)$ untuk sorting sudah optimal dan itulah yang digunakan di dunia nyata.

Checklist Submit

1	<code>01-intro-kompleksitas.js</code> — perbandingan $O(1)$, $O(n)$, $O(n^2)$ empiris	✓
2	<code>02-kelas-big-o.js</code> — contoh semua kelas Big O dan memoization	✓
3	<code>latihan-1.js</code> — identifikasi kompleksitas + two sum benchmark	✓
4	<code>03-space-complexity.js</code> — perbandingan $O(1)$ dan $O(n)$ space	✓

5	<code>04-refactor.js</code> — refactoring nested loop ke Set/Map	☒
6	<code>tugas/tugas-1.js</code> — intersection, anagram, kuadrat	☒
7	<code>tugas/tugas-2.js</code> — benchmark visualisasi pertumbuhan	☒
8	Semua file di-push ke GitHub	☒

Perintah Submit ke GitHub

```
cd ..  
git status  
git add praktikum-5/  
git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"  
git push origin main
```